



Microcontroladores

Unidad 4: Periféricos

4.7: PWM

Dr. Carlos Alberto Aguilar Avelar
Ing. en Computación

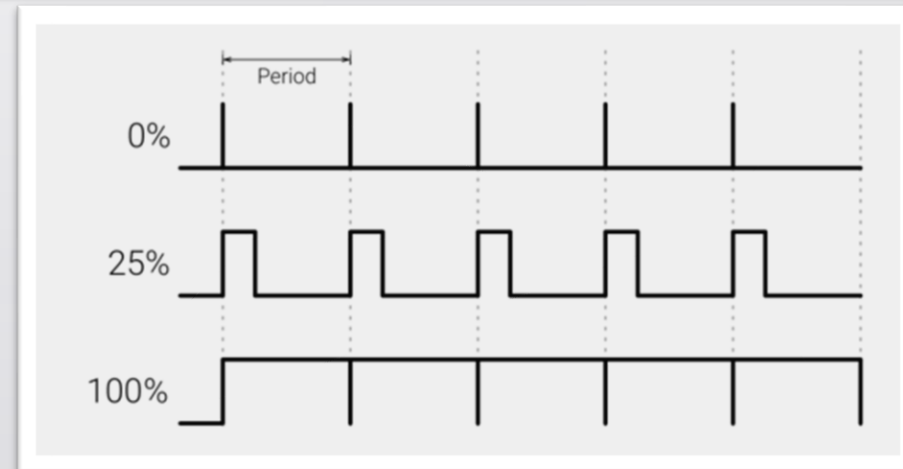
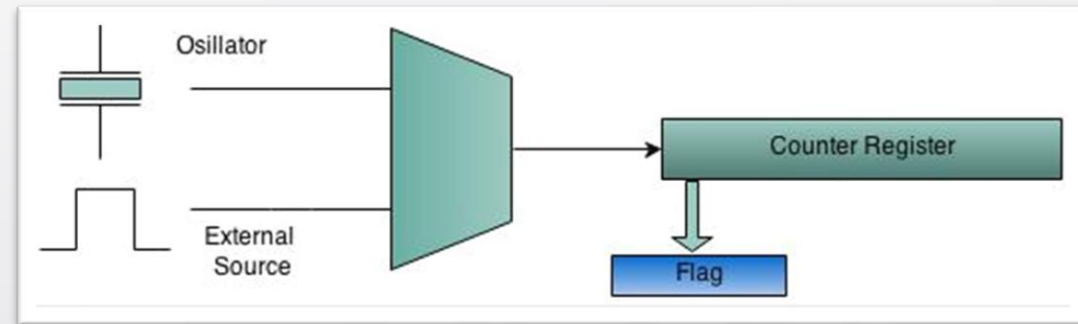
Facultad de Ingeniería
Universidad Autónoma de Baja California

Contadores y Temporizadores (Timers)

Un contador o temporizador es un periférico interno del microcontrolador que es utilizado para contabilizar la ocurrencia de eventos externos o para cronometrar la ejecución de una tarea periódica. El conteo se realiza de manera independiente y paralela a lo ejecutado por la CPU.

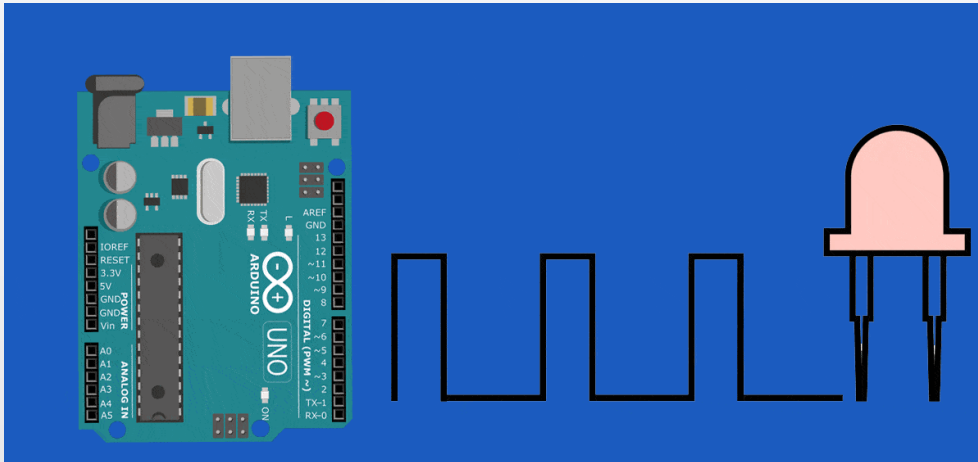
Funciones:

1. **Contador (Counter):** Contador de pulsos externos provenientes de una terminal específica.
2. **Temporizador (Timer):** Contador de pulsos provenientes del oscilador principal.
3. **Captura (Capture):** Registra el periodo de ocurrencia de un pulso externo en una terminal específica.
4. **Modulador de ancho de pulso (PWM):** Generador de funciones cuadradas con ciclo de trabajo variable.



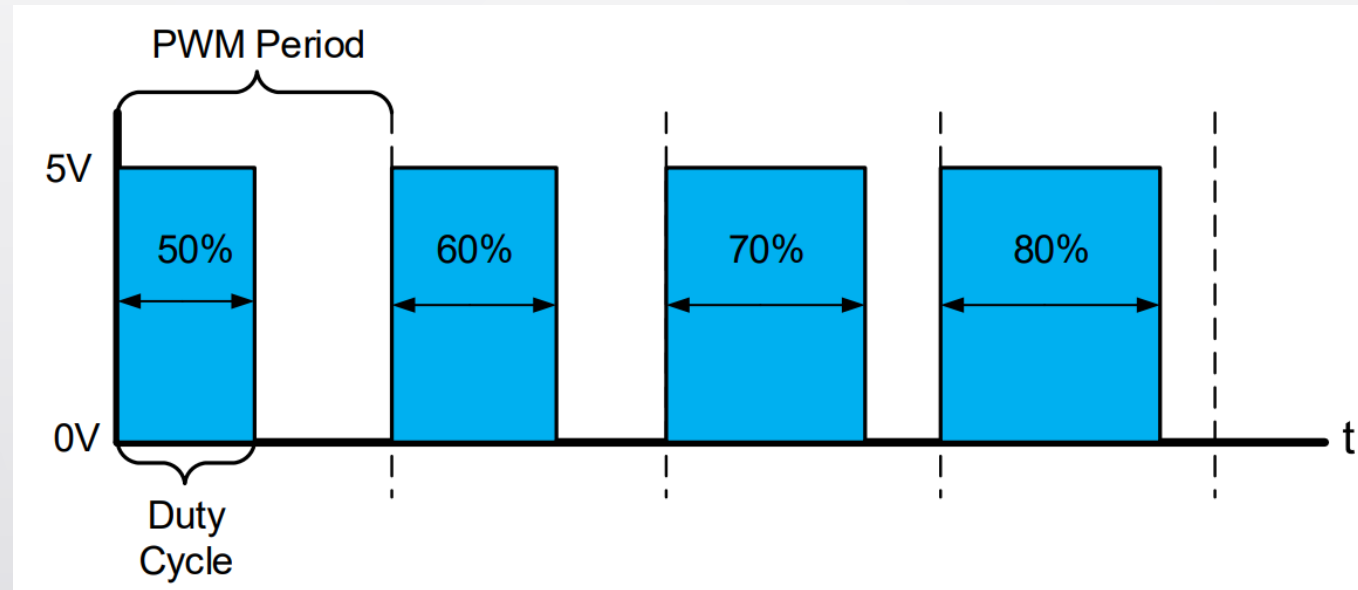
Modulación de Ancho de Pulso (PWM)

La modulación de ancho de pulso es un ejemplo del uso de Timers para la generación de señales periódicas, y es sumamente útil para el control de motores, servomotores y otros actuadores.



Modulación de Ancho de Pulso (PWM)

Las señales moduladas en ancho de pulso son señales digitales de frecuencia fija, cuyo tiempo en el nivel alto es variable. El ciclo de trabajo es la relación del tiempo en alto y el periodo total de la señal portadora, y se expresa en términos de porcentaje.

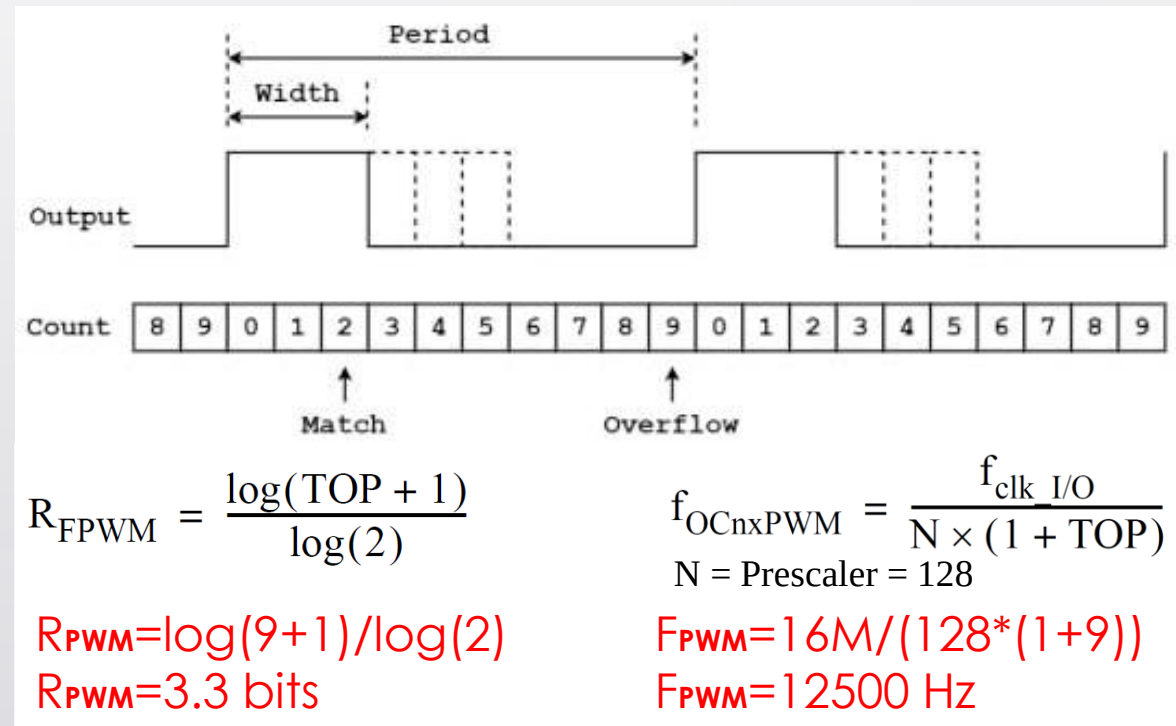


Timers en modo PWM

Para usar un contador como generador de señal **PWM**, se utiliza la salida del **comparador**. La salida se pone a **1** al iniciar el contador y cambia a **0** cuando el valor del contador coincide con el valor del registro de comparación. Por lo tanto, el ciclo de trabajo varía con el valor del registro comparador.

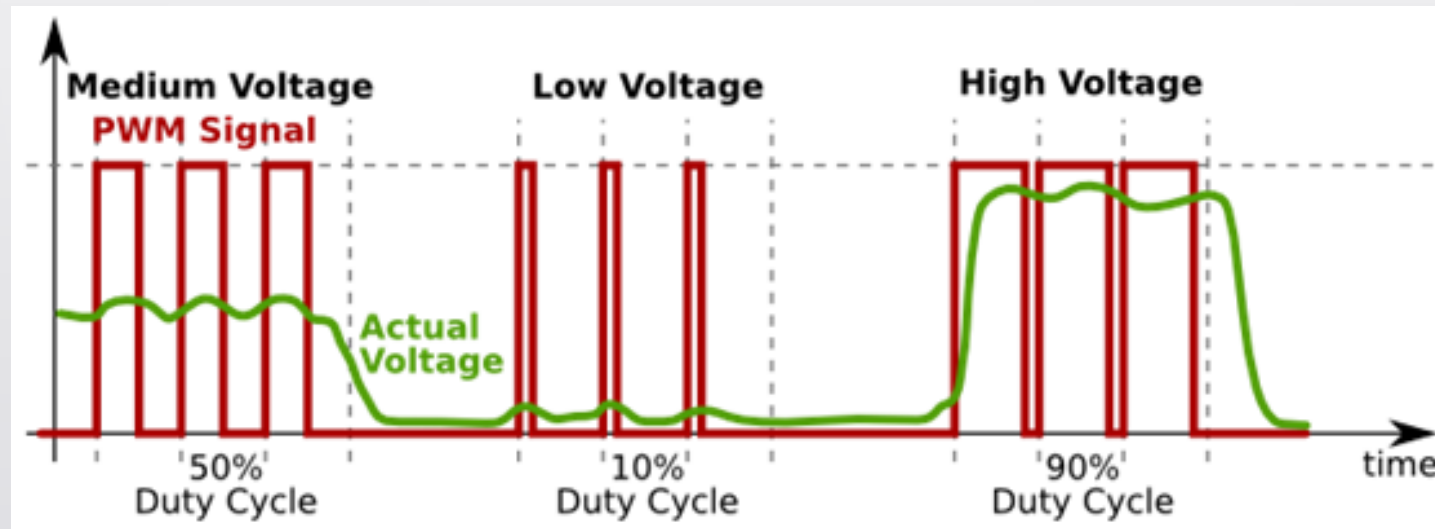
Características:

1. **Frecuencia:** Ciclos por segundo de la señal portadora.
2. **Periodo:** Duración de un ciclo completo de la señal portadora.
3. **Ciclo de trabajo:** Porcentaje de tiempo por ciclo que la señal se encuentra en alto.
4. **Resolución:** Número de pasos o incrementos del ciclo de trabajo. Se puede expresar en bits.



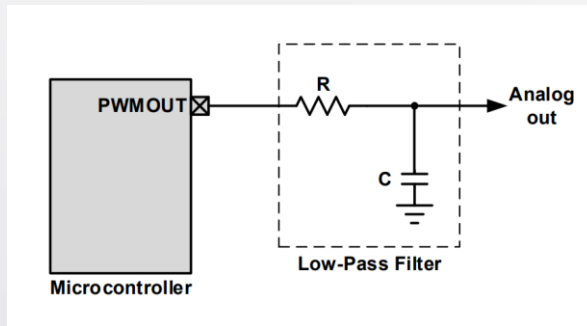
PWM como DAC

Es posible utilizar una salida PWM del microcontrolador como una salida de voltaje analógico. Dicho de otro modo, el módulo PWM puede funcionar como un convertidor digital-analógico (DAC). El voltaje promedio de la señal digital modulada puede ser utilizada por algunos circuitos o componentes electrónicos como una señal de voltaje analógica, aunque esto podría requerir una etapa de filtrado de la señal digital.



PWM como DAC

El valor de voltaje analógico es directamente proporcional al ciclo de trabajo multiplicado por la amplitud de la señal digital. Por ejemplo, para un ciclo de trabajo del 50% y una amplitud digital de 5V, es posible obtener una salida de voltaje analógico de 2.5V, que es igual al voltaje promedio de la señal modulada.



- Es necesario utilizar un filtro pasa-bajas para eliminar los componentes de alta frecuencia de la señal modulada.

$$V_{OUT} = A \times \text{Duty Cycle}$$

where:

$$A = \text{Logic 'high' voltage}$$

$$RC = \frac{1}{2\pi \times f_C}$$

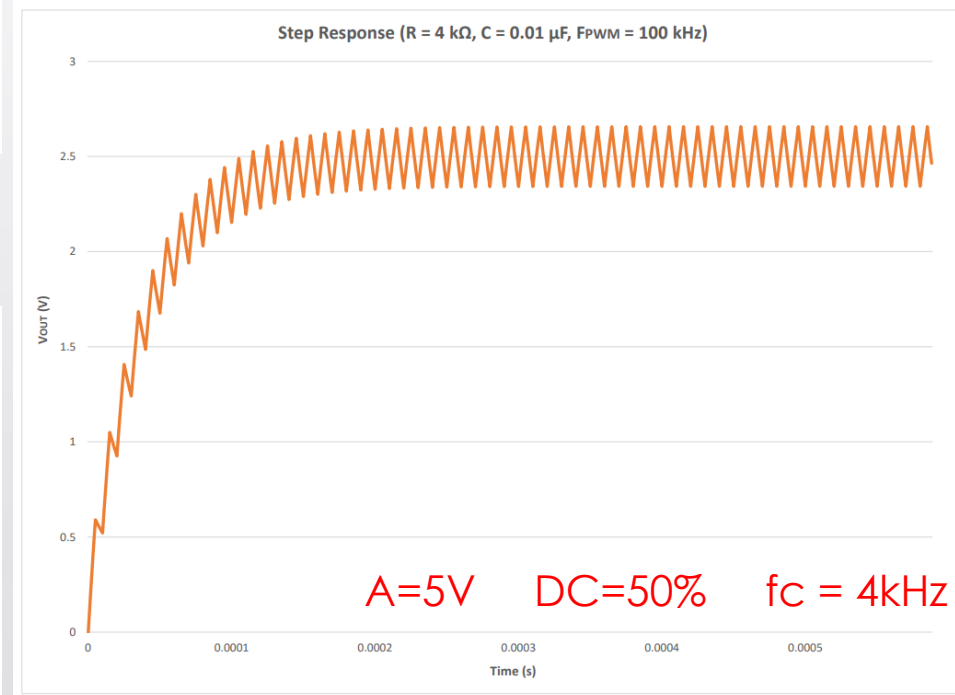
where:

R : Resistance

C : Capacitance

f_C : cut - off frequency

Figure 2-8. Step Response ($R = 4 \text{ k}\Omega$, $C = 0.01 \text{ }\mu\text{F}$, $F_{PWM} = 100 \text{ kHz}$)



PWM en el ATmega328P

El microcontrolador Atmega328P cuenta con 3 módulos independientes de Temporizadores/Contadores.

Timer 0 (8 bits)

- PWM (Output Compare/OC0A-B)

Timer 1 (16 bits)

- PWM (Output Compare/OC1A-B)

Timer 2 (8 bits)

- PWM (Output Compare/OC2A-B)

***Cualquier pin digital puede ser usado como salida PWM emulada.**

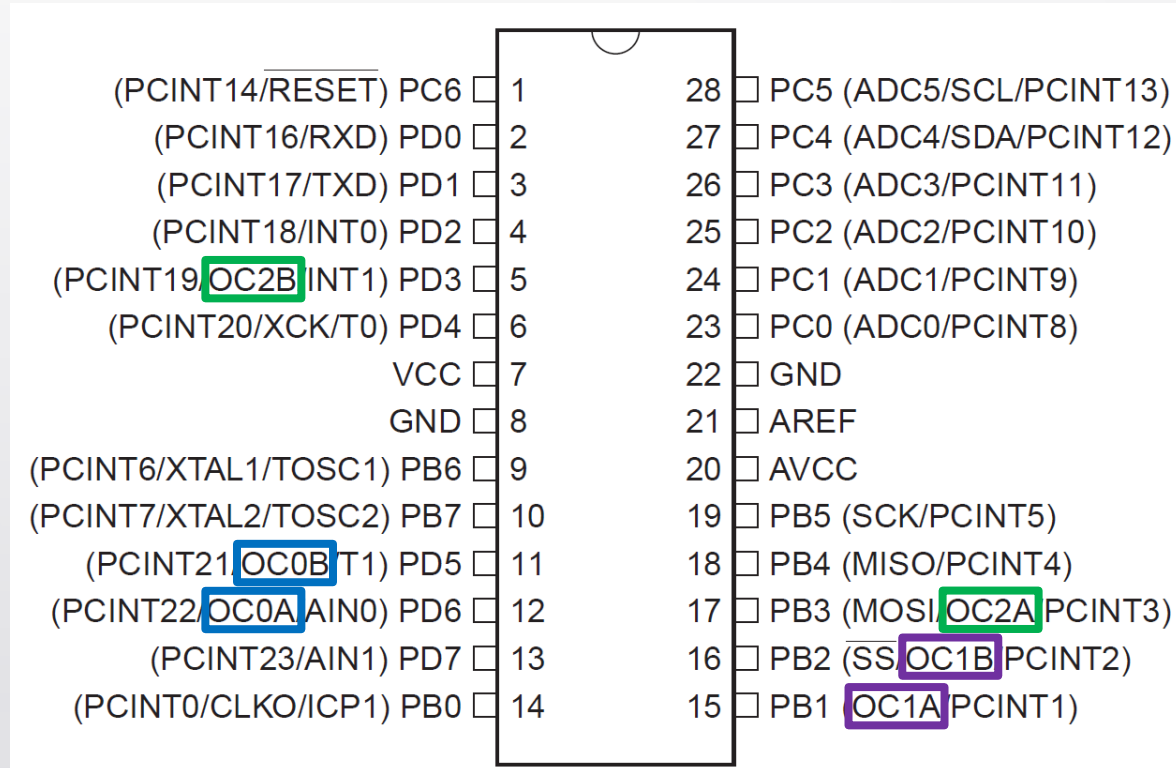
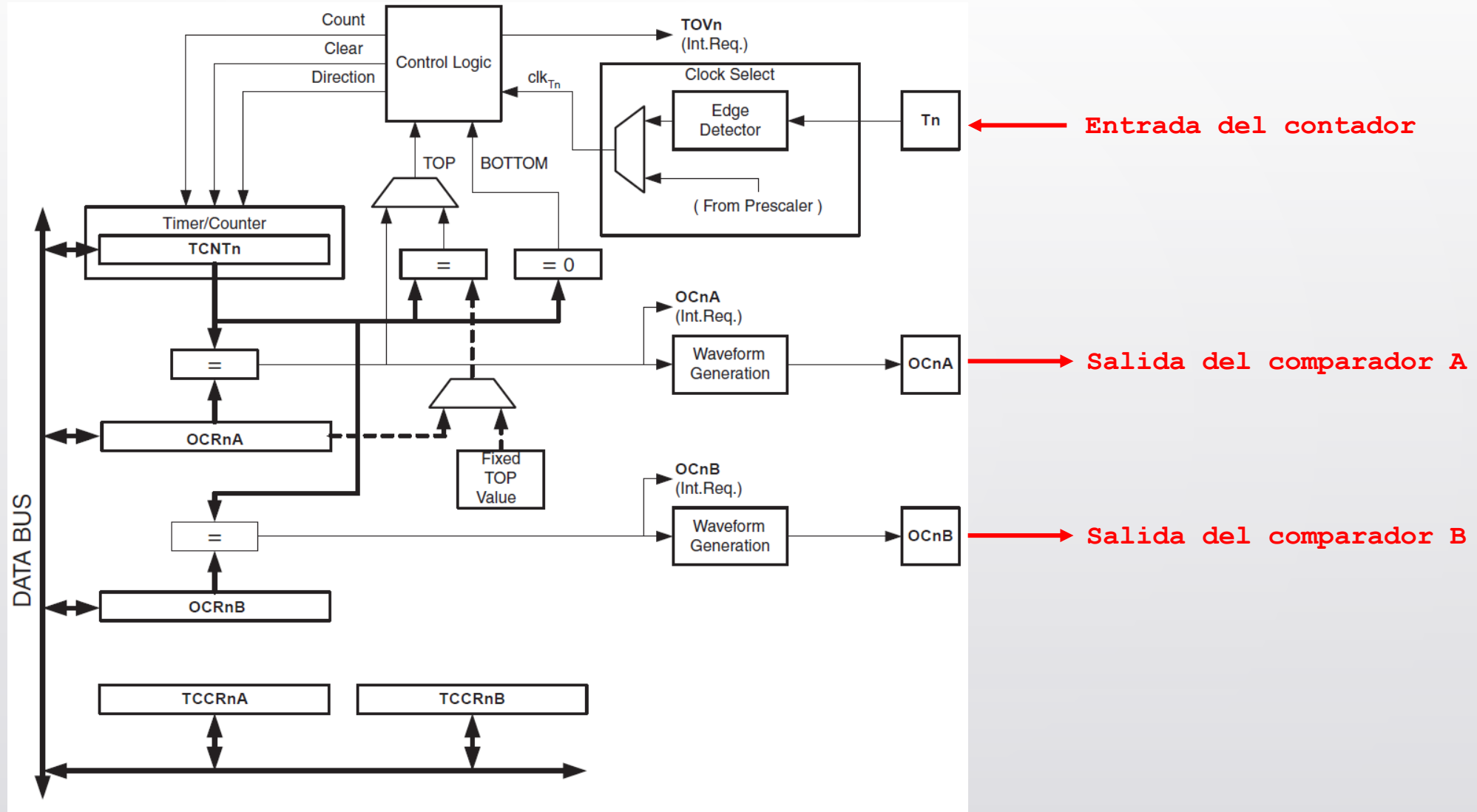


Diagrama del Timer



Registros del Timer: Contador

TIMER 0 (PWM con resolución de 8 bits/255 pasos)

TCNT0 – Timer/Counter Register

Bit	7	6	5	4	3	2	1	0
0x26 (0x46)	TCNT0[7:0]							
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Bits 7-0: TCNT0 Timer/Counter Register (8 bits)

TIMER 1 (PWM con resolución de 16 bits/65535 pasos)

TCNT1H and TCNT1L – Timer/Counter1

Bit	7	6	5	4	3	2	1	0
(0x85)	TCNT1[15:8]							
(0x84)	TCNT1[7:0]							
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Bits 15-0: TCNT1 Timer/Counter Register (16 bits)

Registros del Timer: Compare Registers A/B

OCR0A – Output Compare Register A

Bit	7	6	5	4	3	2	1	0
0x27 (0x47)	OCR0A[7:0]							
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Bits 7-0: OCR0A: OC0A set if OCR0A match with TCNT0

OCR0B – Output Compare Register B

Bit	7	6	5	4	3	2	1	0
0x28 (0x48)	OCR0B[7:0]							
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Bits 7-0: OCR0B: OC0B set if OCR0B match with TCNT0

***2 salidas PWM independientes por cada Timer**

Registros del Timer: Comparar Config.

TCCR0A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Bits 7:6 – COM0A1:0: Compare Match Output A Mode

Bits 5:6 – COM0B1:0: Compare Match Output B Mode

Bits 1:0 – WGM01:0: Waveform Generation Mode

Table 14-3. Compare Output Mode, Fast PWM Mode⁽¹⁾

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected.
0	1	WGM02 = 0: Normal port operation, OC0A disconnected. WGM02 = 1: Toggle OC0A on compare match.
1	0	Clear OC0A on compare match, set OC0A at BOTTOM, (non-inverting mode).
1	1	Set OC0A on compare match, clear OC0A at BOTTOM, (inverting mode).

Table 12-8. Waveform Generation Mode Bit Description

Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on ⁽¹⁾⁽²⁾
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

Registros del Timer: Clock Source Config.

TCCR0B – Timer/Counter Control Register B

TIMER 0	Bit	7	6	5	4	3	2	1	0
	0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00
	Read/Write	W	W	R	R	R/W	R/W	R/W	R/W
	Initial Value	0	0	0	0	0	0	0	0

Bits 2-0: CS0 2-0: Clock Select Bits

Bit 7 – FOC0A: Force Output Compare A

Bit 6 – FOC0B: Force Output Compare B

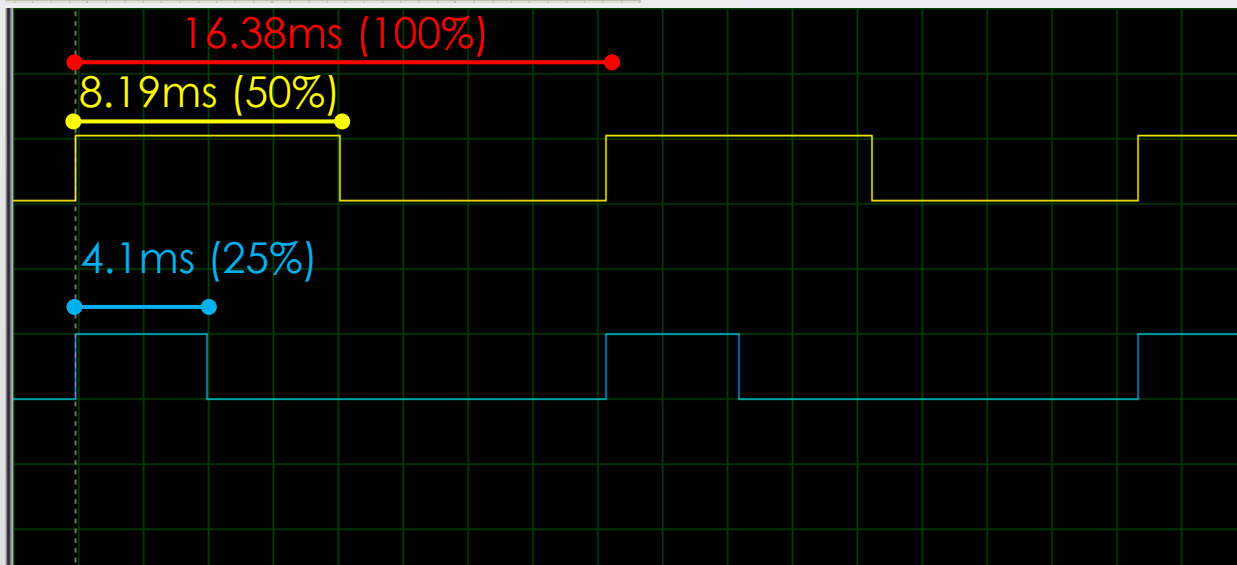
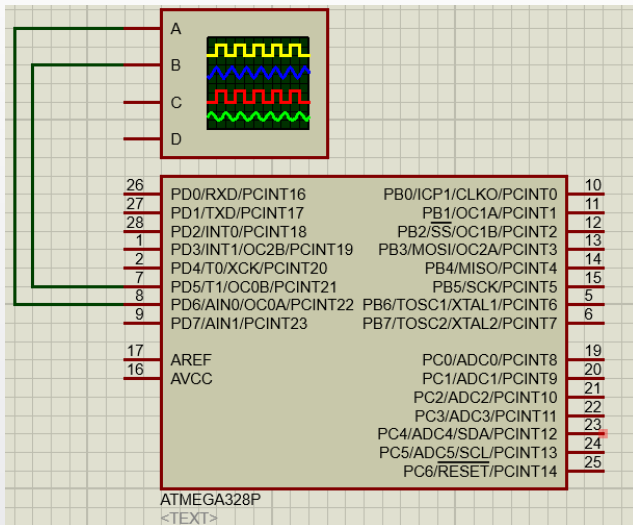
Table 12-9. Clock Select Bit Description

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$\text{clk}_{I/O}$ (No prescaling)
0	1	0	$\text{clk}_{I/O}/8$ (From prescaler)
0	1	1	$\text{clk}_{I/O}/64$ (From prescaler)
1	0	0	$\text{clk}_{I/O}/256$ (From prescaler)
1	0	1	$\text{clk}_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

Table 12-8. Waveform Generation Mode Bit Description

Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on ⁽¹⁾⁽²⁾
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

Ejemplo #1 - 25% and 50% PWM SIGNALS



```
#include <avr/io.h>
```

```
int main(void)
```

```
{
```

```
// DDRD Config.
```

```
DDRD |= 1<<PIND5; // PinD5 as Output
```

```
DDRD |= 1<<PIND6; // PinD5 as Output
```

```
// Timer0 Config. (PWM Non-inverting)
```

```
TCCR0A |= (1<<COM0A1)|(0<<COM0A0);
```

```
TCCR0A |= (1<<COM0B1)|(0<<COM0B0);
```

```
// Fast PWM
```

```
TCCR0A |= (1<<WGM01)|(1<<WGM00);
```

```
// TOP=0xFF
```

```
TCCR0B |= (0<<WGM02);
```

```
// ClockSelect=101: Prescaler 1024
```

```
TCCR0B |= (1<<CS02)|(0<<CS01)|(1<<CS00);
```

```
// Compare value
```

```
OCR0A = 128; //127/255: 50% Duty Cycle
```

```
OCR0B = 64; //63/255: 25% Duty Cycle
```

```
while (1);
```

```
}
```

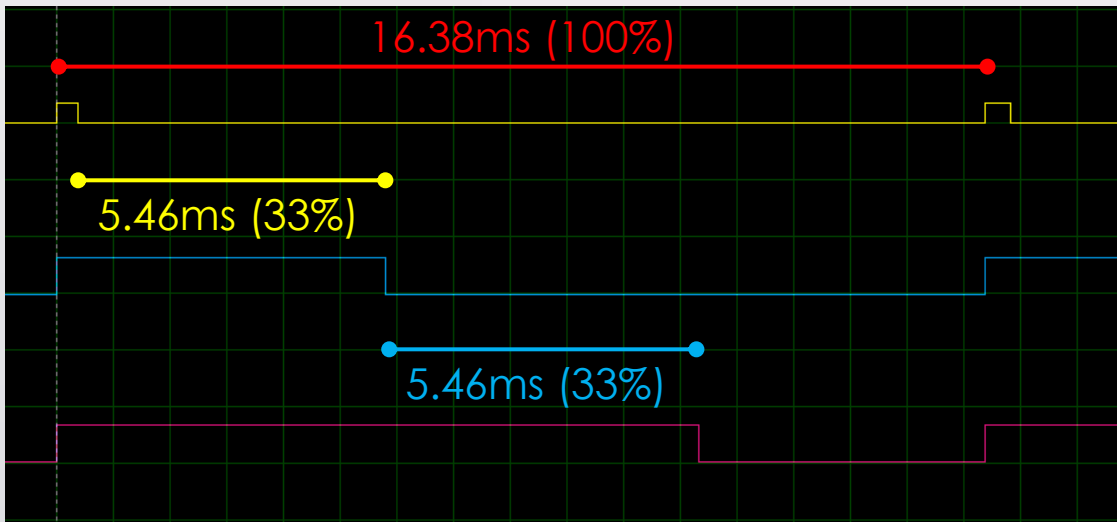
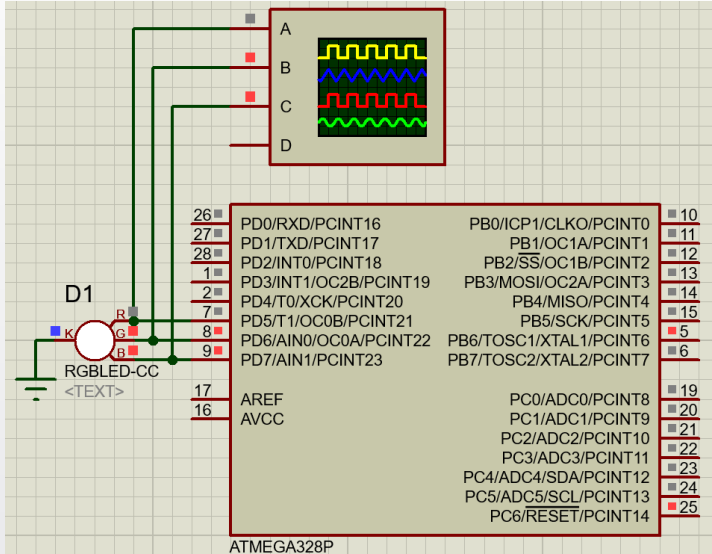
$R_{PWM} = 8 \text{ bits (255 pasos)}$

$F_{PWM} = 16M/(1024*(1+255))$

$F_{PWM} = 61.03 \text{ Hz}$

$T_{PWM} = 16.38ms$

Ejemplo #2- LED RGB (Emulated PWM)



```
#include <avr/io.h>
```

```
int main(void)
```

```
{  
    // PORTD GPIO Config.  
    DDRD |= 1<<PIND5; // PinD5 as Output  
    DDRD |= 1<<PIND6; // PinD6 as Output  
    DDRD |= 1<<PIND7; // PinD7 as Output  
    // Timer0 Config. (PWM Non-inverting)  
    TCCR0A |= (1<<COM0A1)|(0<<COM0A0); //OC0A Mode  
    TCCR0A |= (1<<COM0B1)|(0<<COM0B0); //OC0B Mode  
    TCCR0A |= (1<<WGM01)|(1<<WGM00); //Mode:Fast PWM  
    TCCR0B |= (0<<WGM02); // TOP=0xFF  
    TCCR0B |= (1<<CS02)|(0<<CS01)|(1<<CS00); //Prescaler:1024  
    // Initial Compare Values  
    uint8_t ValueR = 0; // 0% offset  
    uint8_t ValueG = 85; // 33% offset  
    uint8_t ValueB = 170; // 66% offset  
  
    while (1)  
    {  
        if (TIFR0 & (1<<TOV0)) // If TOV0 Flag is set  
        {  
            TIFR0 |= (1<<TOV0); // Clear TOV0 Flag  
            ValueR++; ValueG++; ValueB++;  
            OCR0B = ValueR; // Set OCR0B value  
            OCR0A = ValueG; // Set OCR0A value  
        }  
        // Emulated PWM on PIND7  
        if (TCNT0 > ValueB) PORTD &= ~(1<<PIND7);  
        else PORTD |= (1<<PIND7);  
    }  
}
```


Actividad #4.7- Regulador de velocidad (PWM)

Utilizando una salida PWM del microcontrolador ATmega328P, programe y simule en Proteus un regulador de velocidad para un motor de corriente directa de 12V utilizando un driver para motores L293D/L298N. La simulación debe contar con las siguientes características:

Especificaciones:

- La velocidad se ajusta a 0%, 25%, 50%, 75% y 100% mediante interruptores deslizables.
- La dirección de giro del motor se ajusta con un interruptor deslizable.

